

性能分析：用 msprof 看懂算子行为

目 录

1 引言：优化离不开测量	2
2 msprof op：分析算子的宏观特点	2
2.1 采集命令	2
2.2 关注哪些指标	2
3 msprof op simulator：观察指令流水与同步	3
3.1 采集命令	3
3.2 它不是真实计时工具	3
4 从现象提出可验证的假设	3
4.1 实验方法论	4
5 两个工具的分工	4
6 本章你将学会	4
7 要点速查	4
8 小结	5

1 引言：优化离不开测量

性能优化离不开测量。只有从 profiling 结果找到根据并针对性分析，才能找到更合理有效的优化方向。凭直觉猜瓶颈往往猜错，比如你以为 Vector 计算是瓶颈，profiling 一看发现 MTE2 搬运占了七成时间。昇腾 NPU 对标 NVIDIA 的 Nsight System / Nsight Compute，提供了 **MindStudio** 一族工具，命令行主要体现为 msprof 类指令。

直觉 不妨把 profiling 想成给算子做体检：你不能只看“总耗时”这一个体温数字，还要看血常规（各单元利用率）、X 光片（指令流水图）、心电图（同步事件）。不同检查看不同指标，组合起来才能定位病因。

本章我们先学习两个主要的 profile 指令，再掌握从 profiling 现象提出可验证假设的方法论。

2 msprof op：分析算子的宏观特点

msprof op 对目标算子进行**板端采集**，即真正在 NPU 上运行并采集性能数据。主要的性能分析结果集中在宏观指标，包括算子运行时间、NPU 上各个单元（如 Vector）的利用率、UB 等不同层级缓存间的带宽、L2 Cache 的命中率、不同 AI Core 任务量是否均匀等。

2.1 采集命令

运行以下指令即可进行一次标准的算子性能数据采集：

```
msprof op \
--kernel-name="fused_add_rms_norm" \
--launch-count=1 \
--aic-metrics=Default \
--output=./op_prof \
python3 checker/test_op.py 2
```

将生成目录中的 visualize_data.bin 导入 **MindStudio Insight** 可以查看图形化结果。

2.2 关注哪些指标

分析结果时可以考虑检查以下问题：

1. Block Dim 与各核工作量是否符合 tiling 设计，是否存在空闲核、明显拖尾或负载不均匀。
2. Vector 等运算单元的利用率是否较高。
3. UB 高速缓存的利用带宽是否够高，次一级的 L2 Cache 命中率是否足够。

利用率表示该运算单元处于活跃状态的比例，不一定等同于“有效工作”的比例。多条流水线可以重叠，因此不同单元之间的利用率没有明显关系；单个指标较高或较低也不足以单独证明瓶颈。

直觉 不妨把“利用率”想成“出勤率”：一个员工全天都在工位上（利用率 100%），不等于他都在干有效活，也可能在等上游交接。要判断瓶颈，还得结合他等待的时间和产出的数量。

3 msprof op simulator: 观察指令流水与同步

Simulator 在 CPU 上模拟指定 SoC 的指令执行, 适合检查**指令发射顺序、流水线空泡、SetFlag/WaitFlag 等同步事件和资源冲突**。主要生成的结果是一个流水图 (也叫 Timeline)。

3.1 采集命令

```
msprof op simulator \  
  --soc-version=Ascend910B4 \  
  --kernel-name="fused_add_rms_norm" \  
  --launch-count=1 \  
  --output=./op_sim \  
python3 checker/test_op.py 2
```

由于是 CPU 侧模拟算子在 NPU 上的行为, 所以需要显式指定 `--soc-version` 为 Ascend910B4。

不要尝试模拟真实数据规模。由于是 CPU 模拟, *msprof op simulator* 的性能非常糟糕。模拟时请设置较小的数据规模, 以便在合适的时间和资源使用下完成模拟。它的价值在于看流水线运转, 不在于看绝对耗时。

3.2 它不是真实计时工具

Simulator 的仿真耗时**不能**与 Task Duration(us) 比较, 缓存、带宽竞争和运行时调度也应**以板端结果为准**。它的强项是让你看到指令的发射顺序、流水线空泡和同步停顿, 这些在板端采集中往往被掩盖。

例 同一个计算功能在两种实现下的模拟流水图差异明显: 优化前的流水图中 Vector 利用率低, 存在大量空泡和同步停顿 (上三角形是各种同步 flag); 优化后 Vector 流水更连续, 同步空泡明显减少。这正是 simulator 擅长展示的信息, 也是指导你插屏障、调流水的依据。

4 从现象提出可验证的假设

Profiler 给出的是**证据**, 而不是自动生成的优化结论。可以按“现象, 假设, 实验”的方式推进:

观测现象	可优先检查	下一步实验
MTE2/MTE3 时间较长, Vector 经常等待	搬运粒度过小, 重复读写 GM, 未形成流水	合并搬运或复用 UB 数据, 通过流水开启双缓冲
Vector 流水长时间连续工作	指令数量, 规约层数, 类型转换或高代价指令	优化计算逻辑, 实现更高性能的同样计算功能
各流水线都存在大段空泡	过宽的屏障, Scalar 依赖, tile 太少	用 simulator 对齐空泡与同步事件, 再收窄同步范围
部分核明显更早结束	多核切分不均或尾块集中	调整 Tiling 和 NPU 侧 Init 时的工作分配

观测现象	可优先检查	下一步实验
Roofline 接近带宽上限	算子可能受 GM 带宽约束	优先减少 GM 往返, 而非继续增加 Vector 指令并行度

4.1 实验方法论

每次只改变一个主要因素, 先跑完整正确性测试, 再重复计时。只有当对应指标与端到端耗时同时朝预期方向变化时, 才能较有把握地解释优化收益。

直觉 不妨把优化想成科学实验: 你是实验者, profiler 是仪器, 算子是实验对象。你要做的是“提出假设, 设计实验, 控制变量, 观察结果, 验证或推翻假设”。一次改两三个变量, 出了问题都说不清是谁的功劳或谁的锅。

失败尝试也值得记录。没有带来加速的尝试可以帮助说明原先的瓶颈判断、优化的副作用, 或不同指标之间的取舍。实验报告里写下“试了 X 但没快, 因为 Y”, 和写下“试了 X 快了 20%”一样有价值。

5 两个工具的分工

工具	运行位置	擅长	不能做什么
msprof op	板端 (真实 NPU)	宏观指标: 时间, 利用率, 带宽, L2 命中率, 核间均衡	看不清指令级流水细节
msprof op simulator	CPU 侧模拟	指令发射顺序, 流水空泡, 同步事件, 资源冲突	绝对耗时不可信, 不能看真实带宽竞争

两者是互补关系: 先用 msprof op 看宏观指标定位“哪个单元是瓶颈”, 再用 msprof op simulator 看指令流水找“为什么这个单元有空泡”, 针对性修改后回到 msprof op 验证端到端耗时是否下降。

6 本章你将学会

1. 用 msprof op 进行板端性能采集, 导入 MindStudio Insight 查看图形化结果。
2. 检查 Block Dim、Vector 利用率、UB 带宽、L2 命中率、核间均衡等宏观指标。
3. 用 msprof op simulator 在 CPU 侧模拟指令流水, 观察发射顺序、空泡与同步事件。
4. 说明 simulator 仿真耗时不能与板端 Task Duration 比较, 以及应设置较小数据规模。
5. 按“现象, 假设, 实验”的方法论推进优化, 每次只改变一个因素并验证正确性。

7 要点速查

概念	要点
msprof op	板端采集, 宏观指标 (时间/利用率/带宽/L2)
msprof op simulator	CPU 模拟, 指令流水图, 看空泡与同步

概念	要点
--soc-version	simulator 必须显式指定, 如 Ascend910B4
利用率	出勤率, 非有效工作率, 需结合等待时间判断
两者分工	op 定位瓶颈单元, simulator 找空泡原因
实验方法	一次一个变量, 先正确性后计时, 看指标与耗时同向变化
失败尝试	同样值得记录, 说明副作用与取舍

8 小结

本章从“优化离不开测量”出发, 介绍了昇腾 NPU 上的两个主要 profiling 工具: msprof op 负责板端宏观指标采集, 定位“哪个单元是瓶颈”; msprof op simulator 负责 CPU 侧指令流水模拟, 找“为什么这个单元有空泡”。我们学习了关注 Block Dim、利用率、带宽、L2 命中率等指标的方法, 并掌握了“现象, 假设, 实验”的优化方法论。后续章节将具体讨论如何针对 profiling 发现的瓶颈, 应用搬运计算流水、UB 复用、tiling、规约等优化手段。

讲义基于 HPC101 2025 Lab3.5 实验内容编写